Assignment -20.1

Hall ticket no:2303A510A6

Batch no:02

Task 1 – Password Storage Vulnerability Check

Task:

Analyze an AI-generated signup script for insecure password storage.

Instructions:

- **Prompt AI to generate a Python user registration program.**
- **Check whether passwords are stored in plain text.**
- **Improve the script using hashing (bcrypt or hashlib).**

Expected Output:

- **A secure registration script with hashed password storage.**

Code:

```python
#Generate a Python user registration program that stores username and password.Here's a simple Python user registration program that allows users to create an account by entering a username and
users = {}

def register():
    username = input("Enter a username: ").strip()
    if not username:
        print("Username cannot be empty.")
        return
    if username in users:
        print("Username already exists.")
        return
    password = input("Enter a password: ").strip()
    if not password:
        print("Password cannot be empty.")
        return
    users[username] = password
    print("Registration successful.")

def login():
    username = input("Enter username: ").strip()
    password = input("Enter password: ").strip()
    if users.get(username) == password:
        print("Login successful.")
    else:
        print("Invalid username or password.")

def main():
    while True:
        print("\n1. Register\n2. Login\n3. Quit")
        choice = input("Choose an option: ").strip()
        if choice == "1":
            register()
        elif choice == "2":
            login()
        elif choice == "3":
            break
        else:
            print("Invalid choice.")

if __name__ == "__main__":
    main()
```

Task 2 – Hardcoded Secrets Detection

Task:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- **Ask AI to generate a script that connects to an external API.**
- **Identify if API keys/passwords are written directly in code.**
- **Refactor using environment variables or config files.**

Expected Output:

- **A secure script with secrets managed properly.**

```python
#Generate a Python script that connects to an external API (for example, a weather API). Include username, password
# Insecure version with hardcoded API key
import requests
def get_weather(city):
    api_key = "your_hardcoded_api_key"
    url = f"http://api.weatherapi.com/v1/current.json?key={api_key}&q={city}"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    else:

        return None
# Example usage
weather_data = get_weather("New York")
print(weather_data)
# Secure version using environment variables
import os
import requests
def get_weather_secure(city):
    api_key = os.getenv("WEATHER_API_KEY")
    if not api_key:
        raise ValueError("API key not found. Please set the WEATHER_API_KEY environment variable.")
    url = f"http://api.weatherapi.com/v1/current.json?key={api_key}&q={city}"
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    else:
        return None
# Example usage
if __name__ == "__main__":
    weather_data_secure = get_weather_secure("New York")
    print(weather_data_secure)
```

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

- **Prompt AI to generate a Flask file upload example.**
- **Check for vulnerabilities like uploading executable files.**
- **Add validation for file type, size, and secure filename handling.**

Expected Output:

- **A secure file upload implementation.**

Code:

```python
#Generate a Python Flask application that allows users to upload files. Include basic file upload functionality withou
from flask import Flask, request, redirect, url_for, render_template_string
import os

app = Flask(__name__)
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

UPLOAD_PAGE = '''
<!doctype html>
<title>Upload File</title>
<h1>Upload a File</h1>
<form method=post enctype=multipart/form-data>
    <input type=file name=file>
    <input type=submit value=Upload>
</form>
'''

@app.route('/', methods=['GET', 'POST'])
def upload_file():
        if request.method == 'POST':
                uploaded_file = request.files.get('file')
                if uploaded_file:
                        save_path = os.path.join(app.config['UPLOAD_FOLDER'], uploaded_file.filename)
                        uploaded_file.save(save_path)
                        return f'File uploaded to {save_path}'
        return render_template_string(UPLOAD_PAGE)

if __name__ == '__main__':
        app.run(debug=True)
```

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

- **Ask AI to generate code using os.system() or subprocess.**
- **Identify command injection risks with user input.**
- **Fix using safe subprocess calls and input validation.**

Expected Output:

- **A secure command execution script.**

Code:

```python
#Generate a Python script that safely executes system commands based on user input. Include input validation and
import subprocess
import shlex

def execute_command(user_input):
    # Input validation: only allow specific commands for safety
    allowed_commands = ['ls', 'pwd', 'echo']  # Example allowed commands
    try:
        # Parse the input safely
        args = shlex.split(user_input)
        if not args:
            raise ValueError("Empty command")
        command = args[0]
        if command not in allowed_commands:
            raise ValueError(f"Command '{command}' not allowed")
        # Execute without shell=True to prevent injection
        result = subprocess.run(args, capture_output=True, text=True, timeout=10)
        return result.stdout, result.stderr, result.returncode
    except Exception as e:
        return "", str(e), 1

# Example usage
if __name__ == "__main__":
    user_cmd = input("Enter a command: ")
    stdout, stderr, code = execute_command(user_cmd)
    print(f"Output: {stdout}")
    print(f"Error: {stderr}")
    print(f"Return code: {code}")
```

Task 5 – Secure Input Handling in Web Forms

Task:

Check AI-generated web form code for improper input validation.

Instructions:

- **Ask AI to generate an HTML + Flask feedback form.**
- **Identify missing validation and unsafe rendering.**
- **Fix using server-side validation and sanitization.**

Expected Output:

- **A secure form resistant to injection attacks.**

Code:

```python
#Generate a Flask web application with an HTML feedback form that takes user input and displays it on a webpage.
from flask import Flask, request, render_template_string

app = Flask(__name__)

@app.route('/', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        user_feedback = request.form['feedback']
        return render_template_string('''
            <h1>Feedback Submitted</h1>
            <p>Your feedback: {{ feedback }}</p>
            <a href="/">Submit another feedback</a>
        ''', feedback=user_feedback)
    return render_template_string('''
        <h1>Feedback Form</h1>
        <form method="post">
            <label for="feedback">Enter your feedback:</label><br>
            <textarea name="feedback" id="feedback" rows="4" cols="50"></textarea><br>
            <input type="submit" value="Submit">
        </form>
    ''')

if __name__ == '__main__':
    app.run(debug=True)
```